

CLOUD COMPUTING

Облачные вычислительные сервисы и идентификация машин. Amazon EC2

Nartea N.

Лекция 4 · Информатика и прикладная информатика

План лекции

1. Строительные блоки облачного приложения.
2. Amazon EC2: обзор сервиса.
3. Из чего состоит экземпляр: AMI, тип, хранилище.
4. Сеть, доступ и первый запуск.
5. Жизненный цикл и подключение.
6. Machine identity: как серверу получить права.
7. Кейс Capital One.
8. Модели оплаты EC2.
9. Проектируем первый сервер проекта.

Вопросы для самопроверки (1 из 2)

1. Из каких строительных блоков состоит облачное приложение и почему изучение удобно начинать с вычислений?
2. Какой облачный сервис нужен, чтобы приложение работало круглосуточно и было доступно пользователям?
3. Что означает каждая часть в имени типа экземпляра `m7g.large`?
4. Что произойдёт с данными на `instance store`, если экземпляр остановить и снова запустить?
5. Платите ли вы что-нибудь за остановленный экземпляр?

Вопросы для самопроверки (2 из 2)

6. Почему класть access keys на сервер плохая идея, даже если репозиторий приватный?
7. Как приложение на EC2 получает права, если в его коде и настройках нет ни одного ключа?
8. Что такое Instance Metadata Service и чем IMDSv2 безопаснее IMDSv1?
9. В каких случаях выгодны Spot-экземпляры, а в каких от них лучше отказаться?

Результаты обучения

- Объяснить, из каких строительных блоков собирается облачное приложение.
- Выбрать AMI, тип экземпляра, хранилище и правила Security Group для простого веб-сервера.
- Запустить экземпляр EC2 и подключиться к нему по SSH.
- Объяснить, что происходит с экземпляром и его данными при Reboot, Stop и Terminate.
- Выдать экземпляру права через роль IAM без постоянных ключей.
- Объяснить, чем IMDSv2 защищает от SSRF, и выбрать модель оплаты под нагрузку.

Ретроспектива

- **Глава 2.** Гипервизор делит физический сервер на **виртуальные машины**, и каждая уверена, что работает на своём компьютере.
- **Глава 3.** Облако глазами клиента: регионы и зоны, аккаунт, **пользователь IAM** и ключ доступа для CLI. Коротко упомянули **роли**.
- **Эта глава** соединяет обе темы: запускаем ту самую виртуальную машину как клиенты AWS и выдаём права **не человеку, а серверу**.

01

Строительные блоки облачного приложения

От частей сервера к сервисам облака

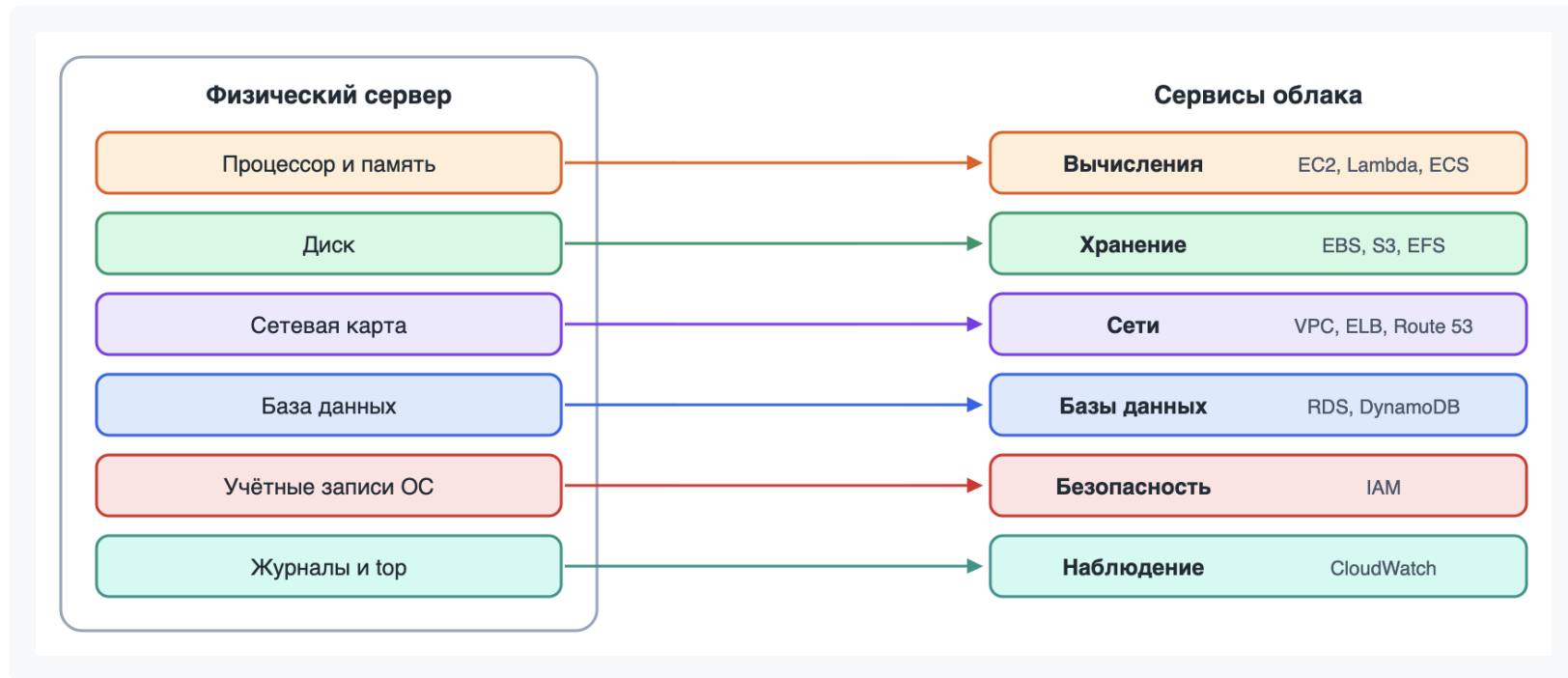


Рис. 4.1. От частей сервера к строительным блокам облака

Строительный блок

Строительный блок (building block) -

группа облачных сервисов, решающих одну базовую задачу, из которых собирается приложение.

Провайдер превращает каждую часть сервера в **отдельный сервис**. Поэтому части можно менять **независимо**: увеличить диск, не трогая процессор, или добавить второй сервер к той же базе данных.

Шесть основных блоков

Блок	Какую задачу решает	Сервисы AWS
Вычисления	Где выполняется код	EC2, Lambda, ECS
Сети	Как части связаны друг с другом	VPC, ELB, Route 53
Хранение	Где лежат диски и файлы	EBS, S3, EFS
Базы данных	Где лежат данные приложения	RDS, DynamoDB
Безопасность	Кто и что может делать	IAM
Наблюдение	Что сейчас происходит с системой	CloudWatch

Седьмой блок, **интеграция** (SQS, SNS, EventBridge), понадобится во второй половине курса.

Путь одного запроса

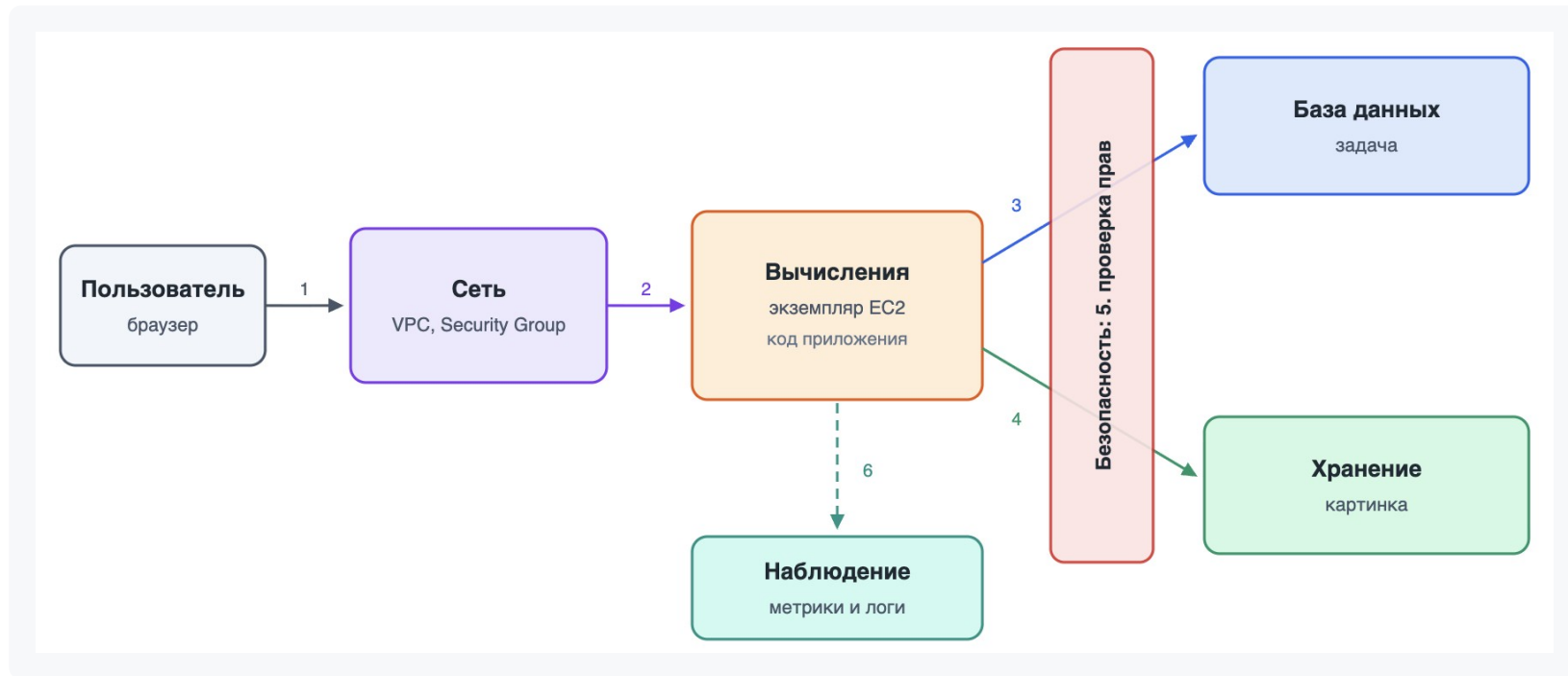


Рис. 4.2. Путь запроса «сохранить задачу с картинкой» через строительные блоки

Почему начинаем с вычислений

- **Самое знакомое понятие.** Сервер вы уже видели во второй главе в виде виртуальной машины. В облаке это та же машина, только создаётся кнопкой.
- **Остальные блоки подключаются к вычислениям.** Диск монтируется к серверу, сеть соединяет серверы, база обслуживает приложение, которое где-то выполняется.
- **Первый сервер можно запустить сегодня.** В каждом регионе есть готовая сеть по умолчанию, глубоко в сети мы погрузимся позже.

02

Amazon EC2: обзор сервиса

Вычисления в облаке

Ноутбук вы закрываете и уносите домой. Приложению нужен компьютер, который **включён круглосуточно** и доступен по сети.

Вычислительный сервис (compute service) -

облачный сервис, который предоставляет процессорное время и память для выполнения вашего кода.

Можно получить **пустую виртуальную машину**, а можно отдать провайдеру **только код**. Разница в **границе ответственности**: чем выше абстракция, тем больше забот у провайдера и меньше свободы у вас.

Amazon EC2

Amazon EC2 (Elastic Compute Cloud) -

сервис AWS для создания и управления виртуальными машинами с оплатой по факту использования.

- **Экземпляр (instance)** - одна виртуальная машина, запущенная в EC2. У неё есть vCPU, память, диск и сетевой интерфейс.
- Под экземпляром работает **гипервизор**, как во второй главе. Только сервер стоит в дата-центре AWS, а вместо установщика у вас кнопка Launch instance.

Amazon Web Services (2026). What is Amazon EC2? Amazon EC2 User Guide.

EC2 коротко

Характеристика	Amazon EC2
Модель	IaaS: ОС и всё, что выше, на вашей ответственности
Область действия	Региональный сервис, каждый экземпляр зональный
Оплата	Время работы экземпляра; диски и публичные IPv4 отдельно
Как работать	Консоль, CLI <code>aws ec2</code> . . . , SDK
Появился	Бета в августе 2006, полноценный сервис с октября 2008
Аналоги	Google Compute Engine, Azure Virtual Machines

Elastic: экземпляр создают за минуту, меняют размер и удаляют, в отличие от купленного сервера.

Вопрос: зона доступности

Приложение работает на одном мощном экземпляре m7g.4xlarge в зоне eu-central-1a. Зона отключилась. Что будет с приложением?

- A. Экземпляр автоматически переедет в eu-central-1b.
- B. Ничего: EC2 региональный сервис.
- C. Приложение станет недоступно.
- D. Мощный тип экземпляра переживёт сбой.

Ответ: зона доступности

A. Экземпляр автоматически переедет в eu-central-1b.

B. Ничего: EC2 региональный сервис.

C. Приложение станет недоступно.

D. Мощный тип экземпляра переживёт сбой.

ОТВЕТ

C. Приложение станет недоступно. EC2 как сервис региональный, но каждый экземпляр физически работает на сервере в **одной** зоне. Мощность не заменяет отказоустойчивость: как это исправить, разберём в главе о сетях и масштабировании.

ЗАПОМНИТЕ

**Один экземпляр, каким бы мощным он ни был,
это одна точка отказа.**

03

Из чего состоит экземпляр

Семь решений при запуске

При запуске консоль задаёт десяток вопросов. Удобнее думать о них как о паспорте машины:

1. **AMI:** что на ней установлено?
2. **Тип экземпляра:** насколько она мощная?
3. **Хранилище:** где лежат её данные?
4. **Сеть (VPC):** в какой сети она живёт?
5. **Security Group:** кто может до неё достучаться?
6. **Пара ключей:** как на неё войти?
7. **User data:** что она сделает при первом включении?

1. AMI

AMI (Amazon Machine Image) -

шаблон, из которого создаётся экземпляр: образ корневого диска с ОС и, при необходимости, предустановленным ПО, плюс настройки запуска.

Представьте, что установка Linux с флешки уже прошла, а результат сохранили в файл. Из одного AMI можно запустить **сколько угодно одинаковых экземпляров**.

1. AMI: какие бывают

- **AMI от AWS.** Amazon Linux, Ubuntu, Windows Server, Red Hat. С них начинают почти всегда.
- **Собственные AMI.** Настроили экземпляр и сохранили диск как образ. Образ с готовым приложением называют **золотым образом (golden image)**.
- **AWS Marketplace.** Образы сторонних компаний с их ПО, часть платные.
- **Community AMI.** Опубликованы другими пользователями. **Осторожно:** вы не знаете, что внутри.

В курсе используем **Amazon Linux 2023**: пользователь `ec2-user`, пакеты ставятся через `dnf`.

2. Тип экземпляра

Тип экземпляра (instance type) -

готовая конфигурация виртуального оборудования: число vCPU, объём памяти, пропускная способность сети, варианты хранилища.

Произвольную комбинацию вроде 3 vCPU и 5 ГБ памяти задать нельзя. Типов **сотни**, но у их имён строгая структура.

2. Как читать имя типа



Рис. 4.3. Как читать имя типа экземпляра

Amazon Web Services (2026). Amazon EC2 instance type naming conventions.

2. Семейства экземпляров

Семейство	Буквы	Для чего
Общего назначения	t, m	Веб-серверы, API, обычные приложения
Для вычислений	c	Нагрузка на CPU: рендеринг, игровые серверы
Для памяти	r, x, z	Много RAM: кеши, in-memory базы
Для хранения	i, d, h	Быстрые локальные диски: NoSQL, Spark
Ускоренные вычисления	p, g, inf, f	GPU, ML-ускорители, FPGA
HPC	hpc	Научные расчёты и моделирование

Запоминать все не нужно: достаточно основных групп и первых букв.

2. Буквы опций после поколения

Буква	Что означает
a	Процессор AMD
g	Процессор AWS Graviton (архитектура ARM)
i	Процессор Intel
d	Есть локальный диск instance store
n	Улучшенная сеть
e	Больше памяти или дискового пространства

Graviton - процессор Amazon на архитектуре ARM: программы должны быть собраны под ARM.

Вопрос: тип экземпляра

Что можно сказать об этом экземпляре, не открывая документацию?

```
r7gd.2xlarge
```

Ответ: тип экземпляра

r7gd.2xlarge

ОТВЕТ

Имя раскладывается на четыре части:

- r - оптимизирован для **памяти**.
- 7 - **седьмое** поколение.
- g и d - процессор **Graviton** и локальный диск **instance store**.
- 2xlarge - **размер**: вдвое больше xlarge.

3. Хранилище: два вида дисков

Amazon EBS (Elastic Block Store) -

сетевое блочное хранилище: виртуальный диск, который подключается к экземпляру по сети и существует независимо от него.

- **Instance store** - временный локальный диск внутри того же физического сервера. Данные на нём **теряются** при остановке и завершении экземпляра.
- Диск с операционной системой называется **корневым томом (root volume)**. Обычно это EBS-том.

3. EBS и instance store

Свойство	EBS	Instance store
Где находится	Отдельная система, по сети	В том же сервере
Переживает остановку	Да	Нет, стирается
Перенести на другой экземпляр	Да, в той же зоне	Нет
Резервные копии	Снимки (snapshots)	Только свои средства
Есть у любого типа	Да	Только у некоторых (d)
Для чего	Корневой том, данные	Кеши, временные файлы

3. EBS и instance store на схеме

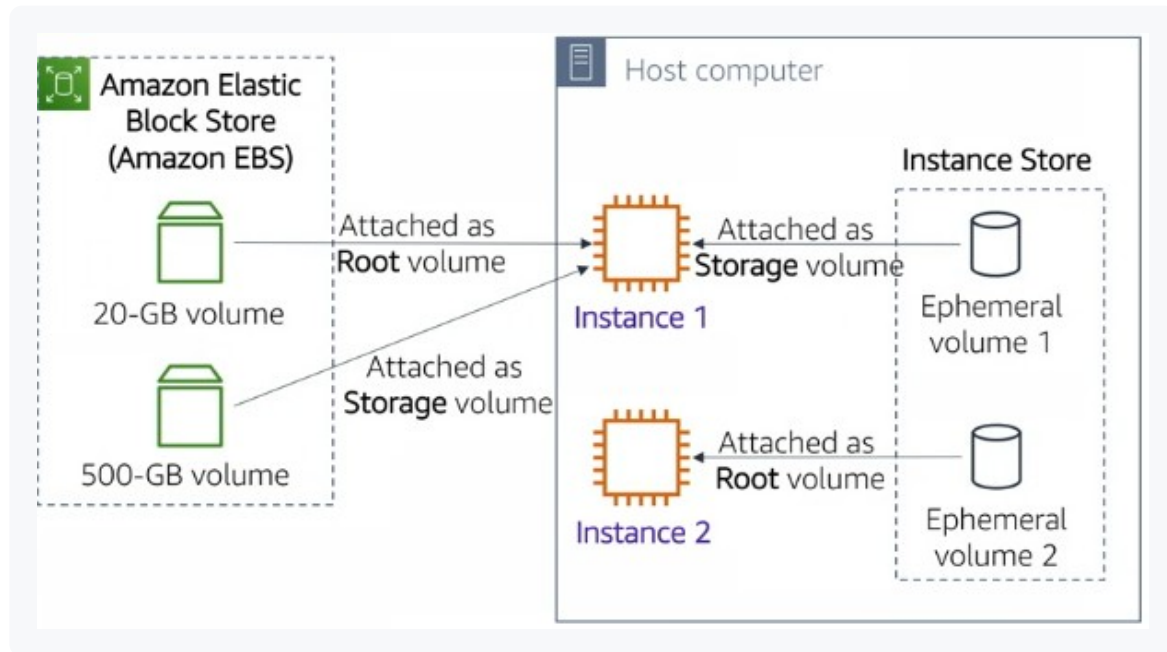


Рис. 4.4. Варианты использования Amazon EBS и instance store

Amazon Web Services (2026). Amazon EC2 User Guide.

Вопрос: хранилище

На `c7gd.large` приложение держит кеш на instance store, а базу SQLite на корневом EBS-томе. Экземпляр остановили и снова запустили. Что осталось?

A. И кеш, и база.

C. Только база на EBS.

B. Только кеш.

D. Ничего не осталось.

Ответ: хранилище

A. И кеш, и база.

B. Только кеш.

C. Только база на EBS.

D. Ничего не осталось.

ОТВЕТ

C. Только база на EBS. Остановленный экземпляр может запуститься уже на другом физическом сервере. EBS-том подключается к нему по сети, а локальный диск остаётся на старом сервере и стирается.

04

Сеть, доступ и первый запуск

4. Сеть экземпляра

VPC (Virtual Private Cloud) -

виртуальная сеть в AWS, в которой запускается экземпляр. Внутри неё подсети, каждая подсеть находится в одной зоне доступности.

- В каждом регионе есть готовая **Default VPC**: в ней можно запускать экземпляры сразу.
- Настройка **Auto-assign Public IP** выдаёт экземпляру публичный адрес для доступа из интернета.

4. Экземпляр в VPC

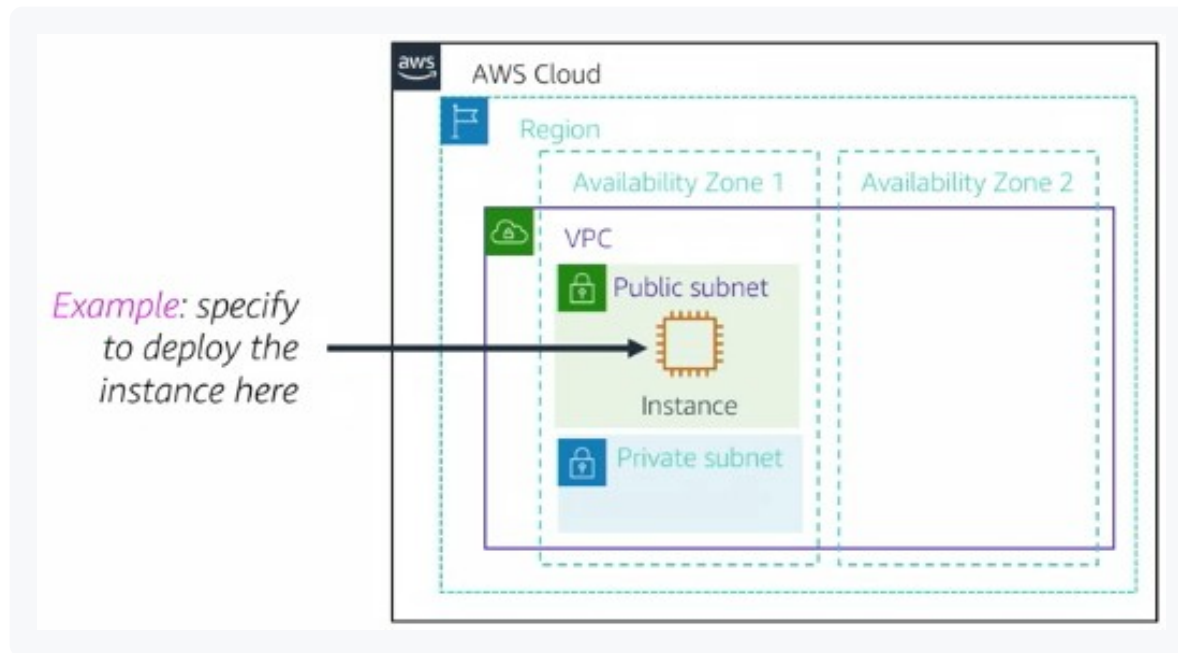


Рис. 4.5. Размещение экземпляра EC2 в публичной подсети VPC

Amazon Web Services (2026). Amazon EC2 User Guide.

4. Публичный IP и Elastic IP

ВАЖНО

Публичный IPv4 **меняется** после остановки и запуска экземпляра. Постоянный адрес даёт **Elastic IP**. С февраля 2024 года AWS берёт почасовую плату за **каждый публичный IPv4**, включая Elastic IP, привязан он к экземпляру или нет.

Barr, J. (2023). New - AWS Public IPv4 Address Charge + Public IP Insights. AWS News Blog.

5. Security Group

Security Group -

виртуальный межсетевой экран на уровне экземпляра, который определяет, какой входящий и исходящий трафик разрешён.

- **Только разрешения.** Правил запрета нет. Новая группа не пропускает ни одного входящего соединения.
- **Исходящий трафик** по умолчанию разрешён весь.
- **Stateful.** Ответ на разрешённый входящий запрос проходит обратно автоматически.

5. Правила для веб-сервера

Протокол	Порт	Источник	Зачем
TCP	22	Ваш IP-адрес, /32	SSH только с вашего компьютера
TCP	80	0.0.0.0/0	HTTP для всех пользователей
TCP	443	0.0.0.0/0	HTTPS для всех пользователей

0.0.0.0/0 означает «любой адрес в интернете». Для портов веб-сервера это нормально, для **порта 22 нет**. Одну группу можно прикрепить к нескольким экземплярам.

Вопрос: Security Group

Какие входящие правила опасны и почему?

Protocol	Port	Source
TCP	80	0.0.0.0/0
TCP	443	0.0.0.0/0
TCP	22	0.0.0.0/0
TCP	3306	0.0.0.0/0

Ответ: Security Group

Protocol	Port	Source
TCP	80	0.0.0.0/0
TCP	443	0.0.0.0/0
TCP	22	0.0.0.0/0
TCP	3306	0.0.0.0/0

ОТВЕТ

Порты 22 и 3306. SSH открывают только со своего IP, иначе боты будут круглосуточно подбирать доступ. 3306 - порт MySQL: база не должна быть видна из интернета. Порты 80 и 443 открыты правильно.

6. Пара ключей: вход по SSH

Боты круглосуточно перебирают пароли, поэтому на Linux-экземпляры AWS входят по **SSH** с ключом, а вход по паролю выключен.

Пара ключей (key pair) -

два связанных файла, публичный и приватный ключ: сервер хранит публичный и пускает только того, у кого есть подходящий приватный.

- **Публичный ключ - замок:** его не страшно показать кому угодно.
- **Приватный ключ - ключ от замка:** есть только у вас и по сети не передаётся.

6. Вход по SSH-ключу

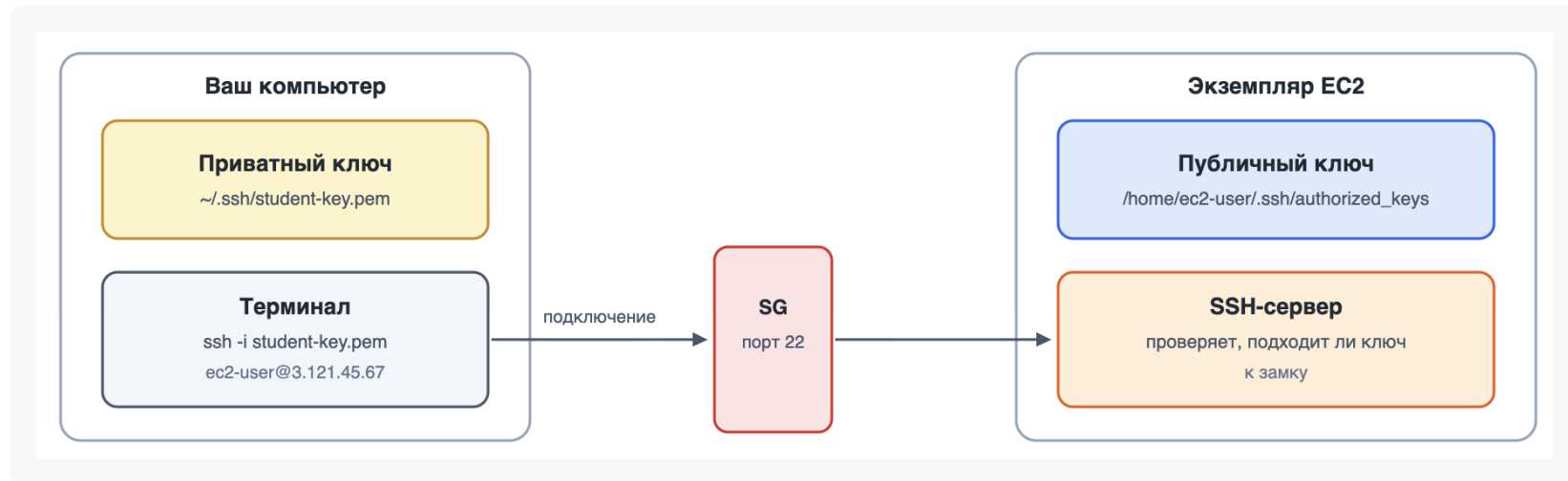


Рис. 4.6. Вход на сервер по SSH-ключу

6. Сохраняем приватный ключ

При создании пары ключей браузер скачает `student-key.pem` **один раз**: AWS хранит только публичную часть.

```
# macOS и Linux
mkdir -p ~/.ssh
mv ~/Downloads/student-key.pem ~/.ssh/
chmod 400 ~/.ssh/student-key.pem
```

`chmod 400` закрывает файл от других пользователей, иначе `ssh` откажется с ним работать. В Windows файл кладут в `C:\Users\<имя>\.ssh\`.

7. User data

User data - скрипт, который экземпляр выполняет **один раз при первом запуске**.

Например, установка nginx:

```
#!/bin/bash
dnf install -y nginx
systemctl enable --now nginx
```

- Выполняет его cloud-init от имени root. При перезагрузках скрипт больше не запускается.
- Скрипт с ошибкой не остановит запуск: экземпляр поднимется без нужного ПО. Причину ищут в /var/log/cloud-init-output.log.

Первый запуск: все решения

Решение	Выбор для учебного веб-сервера
1. AMI	Amazon Linux 2023
2. Тип экземпляра	t3.micro, доступный в бесплатном плане
3. Хранилище	Корневой EBS-том по умолчанию
4. Сеть	Default VPC, публичный IP включён
5. Security Group	SSH со своего IP, HTTP для всех
6. Пара ключей	Новая пара ключей
7. User data	Скрипт установки nginx
+ Теги	Name=web-server: тег Name задают всегда

Запускаем первый экземпляр EC2

1. Запускаем экземпляр в консоли по таблице решений.
2. Открываем публичный IP в браузере и видим страницу nginx.
3. Подключаемся по SSH и выполняем `hostname`.

Пошаговая инструкция со снимками экрана - в практическом материале. Регион `eu-central-1`.

То же самое из командной строки

Консоль и CLI вызывают один и тот же API: поля формы превращаются в параметры команды.

```
aws ec2 run-instances \  
  --image-id ami-0123456789abcdef0 \  
  --instance-type t3.micro \  
  --count 1 \  
  --security-group-ids sg-0abc12d345ef67890 \  
  --key-name student-key  
  
aws ec2 stop-instances --instance-ids i-0123456789abcdef0  
aws ec2 terminate-instances --instance-ids i-0123456789abcdef0
```

Все параметры и команды: `aws ec2 help`.

05

Жизненный цикл и подключение

Состояния экземпляра

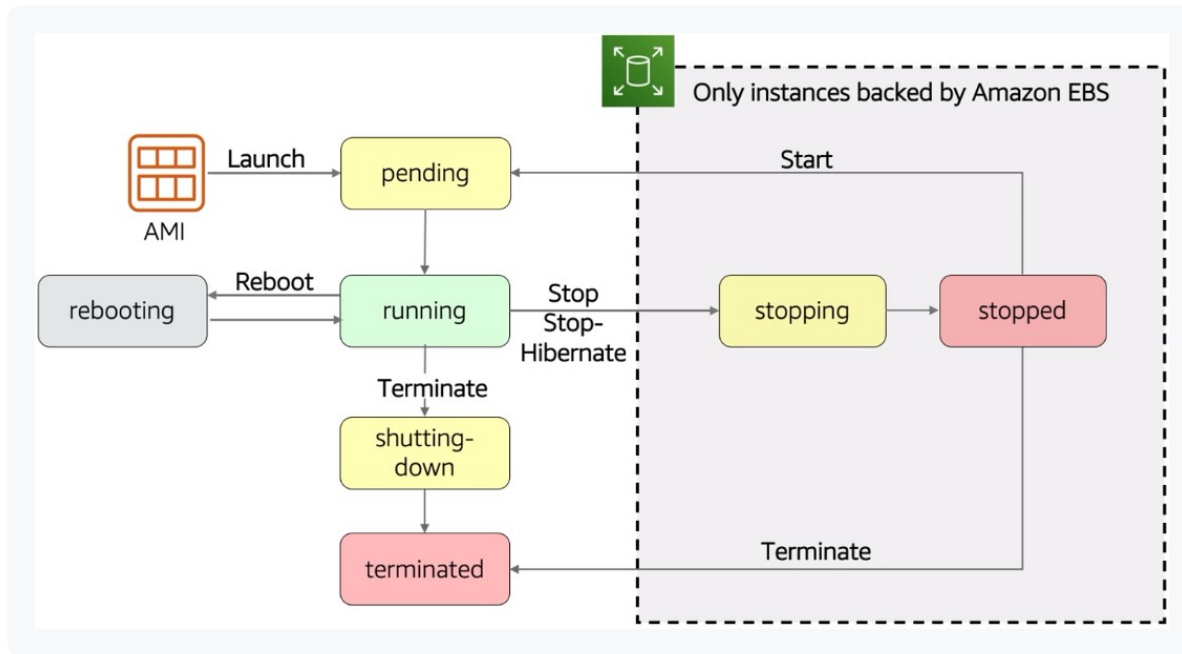


Рис. 4.7. Жизненный цикл экземпляра EC2

Amazon Web Services (2026). Amazon EC2 instance state changes. Amazon EC2 User Guide.

Что переживает экземпляр

Что	Reboot	Stop	Hibernate	Terminate
Оперативная память	Стирается	Стирается	Сохраняется на EBS	Стирается
Данные на EBS	Сохраняются	Сохраняются	Сохраняются	Корневой том удаляется
Данные instance store	Сохраняются	Стираются	Стираются	Стираются
Приватный IP	Сохраняется	Сохраняется	Сохраняется	Освобождается
Публичный IP	Сохраняется	Меняется	Меняется	Освобождается

Stop и Hibernate есть только у экземпляров с корневым томом **EBS**.

Остановка и запуск экземпляра

1. Запишите публичный IP и создайте файл в домашней папке.
2. Остановите экземпляр из Hands-on 4.1 и запустите снова.
3. Сравните IP до и после. Пережил ли остановку файл?

Сначала запишите свой прогноз, потом нажимайте Start.

Вопрос: оплата остановленного экземпляра

Студент добросовестно останавливает экземпляр после каждой пары. Почему в конце месяца в счёте всё равно есть расходы?

Ответ: оплата остановленного экземпляра

ОТВЕТ

Плата за сам экземпляр идёт только в состоянии `running`. Но у остановленного экземпляра остаются платные ресурсы:

- **EBS-тома**: оплачиваются за гигабайты, пока существуют.
- **Elastic IP**: оплачивается, даже если ни к чему не привязан.
- **Снимки EBS и свои AMI**: тоже занимают место.

Если машина больше не нужна, её **удаляют**, а не останавливают.

Подключение по SSH

```
ssh -i ~/.ssh/student-key.pem ec2-user@3.121.45.67
```

```
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
[ec2-user@ip-172-31-20-15 ~]$
```

- `-i` - какой приватный ключ использовать.
- `ec2-user` - пользователь в Amazon Linux (в Ubuntu `ubuntu`).
- `3.121.45.67` - текущий публичный IP экземпляра.
- При первом подключении отвечаем `yes`, выход - `exit`.

Вопрос: подключение по SSH

Команда долго думает и завершается так. Что проверить в первую очередь?

```
ssh: connect to host 3.121.45.67 port 22: Connection timed out
```

- A. Права на файл ключа (`chmod 400`).
- B. Правило для порта 22 и текущий IP.
- C. Имя пользователя `ec2-user`.
- D. Путь к файлу ключа после `-i`.

Ответ: подключение по SSH

A. Права на файл ключа (`chmod 400`).

B. Правило для порта 22 и текущий IP.

C. Имя пользователя `ec2-user`.

D. Путь к файлу ключа после `-i`.

ОТВЕТ

B. Security Group и адрес. Timeout означает, что пакеты **не доходят** до сервера: порт 22 закрыт для вашего IP, или адрес сменился после остановки. Ошибки ключа и пользователя выглядят иначе: сервер отвечает, но не пускает.

Частые ошибки SSH

Сообщение	Что означает	Что сделать
Connection timed out	Пакеты не доходят до сервера	Проверить порт 22 в Security Group и текущий IP
Permission denied (publickey)	Сервер не принял ключ	Проверить имя пользователя и файл ключа
UNPROTECTED PRIVATE KEY FILE!	Файл ключа читают другие	Выполнить <code>chmod 400</code> для ключа
No such file or directory	ssh не нашёл файл ключа	Проверить путь после <code>-i</code>

Подключение без постоянного ключа

EC2 INSTANCE CONNECT

- Кнопка Connect в консоли, терминал в браузере.
- AWS кладёт на экземпляр **временный ключ на 60 секунд**.
- Порт 22 должен быть открыт для сервиса.

SESSION MANAGER

- Терминал в браузере через **агента** на экземпляре.
- **Входящие порты не нужны**, даже 22.
- Агенту нужны права. Как их выдать, узнаем в следующем разделе.

Amazon Web Services (2026). Connect to a Linux instance using EC2 Instance Connect. Amazon EC2 User Guide.

06

Machine identity: как серверу получить права

Вопрос: ключ доступа на сервере

Разработчик положил свой постоянный ключ доступа в `.env` на сервере и в репозиторий, чтобы приложение могло писать в S3. Назовите как можно больше проблем, даже если репозиторий приватный.

Ответ: ключ доступа на сервере

ОТВЕТ

Проблем минимум четыре:

- **Ключ живёт вечно:** утёк, и злоумышленник пользуется им месяцами.
- **Ключ расползается:** .env, репозиторий, бэкапы, золотой образ на сотню машин.
- **Ключ несёт чужие права:** приложению нужна одна папка, а ключ разрешает всё, что можно владельцу.
- **Ключ трудно менять:** нужно обновить на каждом сервере, и какой-нибудь забудут.

Утечки случаются постоянно

- Исследование 2019 года: утёкшие секреты в **100 000+ репозиториях** GitHub, **тысячи новых** в день.
- AWS сама сканирует GitHub: найденный ключ получает политику `AWSCompromisedKeyQuarantineV3`.

ПРЕДУПРЕЖДЕНИЕ

Приватный репозиторий не спасает: его делают публичным, клонируют на ноутбуки, открывают новым людям. Постоянного ключа не должно быть **ни в коде, ни в репозитории, ни на сервере**.

Human identity и machine identity

HUMAN IDENTITY

Учётная запись, от имени которой в облаке действует **человек**.

- Входит в консоль, запускает CLI.
- Может пройти **MFA**.

MACHINE IDENTITY

Учётные данные, от имени которых в облако обращается **программа**.

- Серверов сотни, их создают и удаляют автоматически.
- Ввести код MFA не могут.

AWS и так знает про свой экземпляр всё. Достаточно сказать: «**этому экземпляру разрешено вот это**».

Роль для экземпляра

IAM role (роль IAM) -

идентичность в IAM с набором прав, которая не принадлежит конкретному человеку и не имеет постоянных ключей; её временно принимает тот, кому она нужна.

- **Политика доверия (trust policy):** кто может принять роль. Для роли экземпляра это сервис EC2.
- **Политика прав (permissions policy):** что разрешено. Например, только читать S3.

Политика доверия для EC2

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

Разрешить сервису EC2 принимать эту роль. Руками писать не нужно: выберите AWS service и EC2, консоль составит документ сама. Права прикрепляем готовой политикой, например AmazonS3ReadOnlyAccess.

Временные учётные данные

AWS STS (Security Token Service) -

сервис AWS, который выдаёт временные учётные данные тому, кому разрешено принять роль.

- `AccessKeyId`: временный начинается с ASIA, постоянный ключ пользователя с AKIA.
- `SecretAccessKey` подписывает запросы, без `SessionToken` временный ключ не работает.
- `Expiration`: через несколько часов ключ **перестает работать сам**. EC2 обновляет ключи заранее.

Amazon Web Services (2026). IAM roles for Amazon EC2. Amazon EC2 User Guide.

Вопрос: постоянные и временные ключи

В логах нашли два утёкших идентификатора ключей. Какой опаснее?

- 1) AKIAIOSFODNN7EXAMPLE
- 2) ASIATIOSFODNN7EXAMPLE

A. Первый.

B. Второй.

C. Одинаково опасны.

D. По виду не понять.

Ответ: постоянные и временные ключи

A. Первый.

B. Второй.

C. Одинаково опасны.

D. По виду не понять.

ОТВЕТ

A. Первый. AKIA - постоянный ключ пользователя: работает, пока его не удалят. ASIA - временный ключ роли: через несколько часов он превратится в бесполезную строку.

Instance profile

Instance profile -

контейнер для роли IAM, через который роль передаётся экземпляру EC2.

- Консоль создаёт профиль **с тем же именем**, что и роль. В списке IAM instance profile вы увидите имя роли.
- К экземпляру - **одна роль**, одну роль - к любому числу экземпляров.
- Роль можно прикрепить или заменить **без перезапуска** экземпляра.
- В CLI роль и профиль создаются **двумя командами**: частая ошибка.

Instance Metadata Service

Instance Metadata Service (IMDS) -

служба внутри каждого экземпляра по адресу 169.254.169.254, которая сообщает ему сведения о нём самом: идентификатор, зону доступности, IP-адреса и временные ключи роли.

Это адрес из специального link-local диапазона: запрос **не уходит в интернет**, его обрабатывает сама платформа EC2.

Спрашиваем IMDS изнутри экземпляра

```
IMDS=http://169.254.169.254/latest
TOKEN=$(curl -s -X PUT "$IMDS/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  $IMDS/meta-data/iam/security-credentials/photo-app-role
```

```
{ "Code": "Success", "AccessKeyId": "ASIA...", "SecretAccessKey": "...",
  "Token": "...", "Expiration": "2026-09-21T18:42:10Z" }
```

ответ

Сначала получаем **токен сессии** (так работает IMDSv2), затем временные ключи роли.

Как это работает целиком

1. **Создаём роль** с минимальными правами. Не AdministratorAccess!
2. **Прикрепляем роль к экземпляру**. Ключей не создаём и никуда не копируем.
3. **AWS выдаёт временные ключи** и отдаёт их через IMDS.
4. **SDK забирает ключи** из IMDS и подписывает запросы. AWS проверяет политику роли.
5. **Ключи обновляются заранее**, приложение работает без перезапуска.

ГЛАВНАЯ МЫСЛЬ РАЗДЕЛА

**Права серверу выдаёт роль. Ключи для неё
AWS создаёт сама, и живут они всего
несколько часов.**

07

Кейс Capital One

Capital One, 2019

Раз ключи роли лежат по известному адресу, можно ли заставить сервер отдать их наружу?

- Утекли данные около **106 млн человек**: около 100 млн в США и 6 млн в Канаде.
- Около **30 ГБ** данных из **более чем 700 бакетов S3**.
- Банк узнал об утечке из письма в программу responsible disclosure: данные лежали на GitHub.
- Регулятор OCC оштрафовал банк на **80 млн долларов**.

AWS взломана не была. Как злоумышленница получила ключи?

Novaes Neto, N. et al. (2020). A Case Study of the Capital One Data Breach. MIT Sloan. · OCC (2020). News Release 2020-101.

Вопрос: кейс Capital One

Как злоумышленница получила ключи? У банка в AWS работал экземпляр с межсетевым экраном для веб-приложений (**WAF**), к экземпляру была прикреплена роль.

- A.** Нашла постоянный ключ сотрудника в публичном репозитории.
- B.** Подобрала пароль root-пользователя аккаунта.
- C.** Заставила WAF запросить 169.254.169.254 и вернуть ответ.
- D.** Взломала гипервизор и прочитала память соседней машины.

Ответ: кейс Capital One

- A. Нашла постоянный ключ сотрудника в публичном репозитории.
- B. Подобрала пароль root-пользователя аккаунта.
- C. Заставила WAF запросить 169.254.169.254 и вернуть ответ.**
- D. Взломала гипервизор и прочитала память соседней машины.

ОТВЕТ

C. Через IMDS. WAF был настроен неправильно и выполнял HTTP-запросы по адресу, который подсовывал атакующий. Сервер сам сходил к метаданным и вернул ей временные ключи своей роли.

Как называется такая уязвимость

SSRF (Server-Side Request Forgery) -

уязвимость, при которой сервер можно заставить выполнить HTTP-запрос по адресу, который выбрал атакующий.

Сервер делает запрос **от своего имени и изнутри своей сети**, поэтому достаёт то, что снаружи недоступно. Например, IMDS.

Цепочка атаки

1. Экземпляр с WAF был **неправильно настроен** и уязвим к SSRF.
2. Запрос через WAF к `.../meta-data/iam/security-credentials/` вернул **имя роли**, следующий - её **временные ключи** (IMDSv1 отвечал на обычный GET).
3. С этими ключами злоумышленница работала **со своего компьютера**: ключи роли действуют где угодно, пока не истекут.
4. У роли было **намного больше прав**, чем нужно WAF: список более 700 бакетов и копирование данных.

Novaes Neto, N. et al. (2020). A Case Study of the Capital One Data Breach. MIT Sloan.

Вопрос: кейс Capital One

Чья это ошибка, AWS или банка? Какие две настройки остановили бы атаку или сделали её почти безвредной?

Ответ: кейс Capital One

ОТВЕТ

Ошибки на стороне клиента. Облако сработало ровно так, как было настроено: это модель разделённой ответственности в действии.

- **Наименьшие привилегии:** WAF не нужен доступ ко всем бакетам, ущерб был бы минимальным.
- **Защищённый доступ к IMDS:** простой SSRF-запрос не должен получать ключи. Для этого AWS выпустила IMDSv2.

IMDSv1 и IMDSv2

IMDSv1

- Обычный GET без подготовки.
- Любой SSRF с GET достаёт ключи.

IMDSv2

- Сначала PUT за **токеном**, потом токен в каждом запросе.
- С X-Forwarded-For токен не выдаётся.
- Токен проходит только **1 сетевой хоп**.

Типичный SSRF не умеет отправить PUT с нужными заголовками и упирается в стену.

Уроки кейса

- **Требуйте IMDSv2.** Проверяйте настройку `HttpTokens = required`. В образах Amazon Linux 2023 она включена по умолчанию.
- **Минимум прав роли.** Не `AdministratorAccess` и не доступ ко всем бакетам, а только нужный бакет и нужные действия.
- **Защита в глубину.** Одна ошибка настройки не должна сразу превращаться в утечку данных.

08

Модели оплаты EC2

Одна машина, разные обещания

- **On-Demand.** Оплата посекундно, без обязательств. Самый дорогой за час, но самый гибкий.
- **Reserved Instances.** Резервируете **конфигурацию** на 1 или 3 года и получаете скидку.
- **Savings Plans.** Обещаете тратить **сумму в час** 1 или 3 года. Скидка до 72%.
- **Spot Instances.** Свободные мощности AWS до 90% дешевле, но AWS может **забрать экземпляр**, предупредив за 2 минуты.

Реже: **Dedicated Hosts** и **Dedicated Instances**, выделенное оборудование под лицензии и требования регуляторов.

Amazon Web Services (2026). Amazon EC2 Pricing; Spot Instance interruption notices.

Reserved Instances и Savings Plans

RESERVED INSTANCES

- Обещаете **конфигурацию**: тип, регион, ОС.
- Перешли с `m7g.large` на `c7g.xlarge` или сменили регион: скидка может не распространиться.

SAVINGS PLANS

- Обещаете **сумму в час**, например 10 долларов.
- Compute Savings Plans действуют на любые семейства и регионы, а также на Fargate и Lambda.

Для новых проектов AWS рекомендует **Savings Plans**: скидки те же, гибкости больше.

Модель оплаты под сценарий

Сценарий	Модель	Почему
Сайт факультета, круглосуточно, минимум 3 года	Savings Plans	Постоянная предсказуемая нагрузка
Ночная перекодировка видео, задачу можно повторить	Spot	Прерывание не страшно, цена минимальна
Хакатон на выходные	On-Demand	Коротко, без обязательств
Лицензия ПО привязана к физическим ядрам	Dedicated Hosts	Контроль над физическим сервером

09

Проектируем первый сервер проекта

От требований к решениям

Требование	Решение	Компромисс
Доступ из интернета	Публичный IP, Security Group с портами 80 и 443	Адрес меняется при остановке
Маленькая нагрузка, учебный бюджет	t3.micro или t4g.micro, On-Demand	При долгой нагрузке кончатся CPU credits
Администрирование	SSH по ключу только со своего IP	При смене сети обновить правило
Приложение пишет картинки в S3	Роль IAM с доступом к одному бакету, IMDSv2	Права надо продумать заранее
Быстро пересоздавать сервер	Установка через user data	Первый запуск дольше, чем из golden image

Архитектура учебного сценария

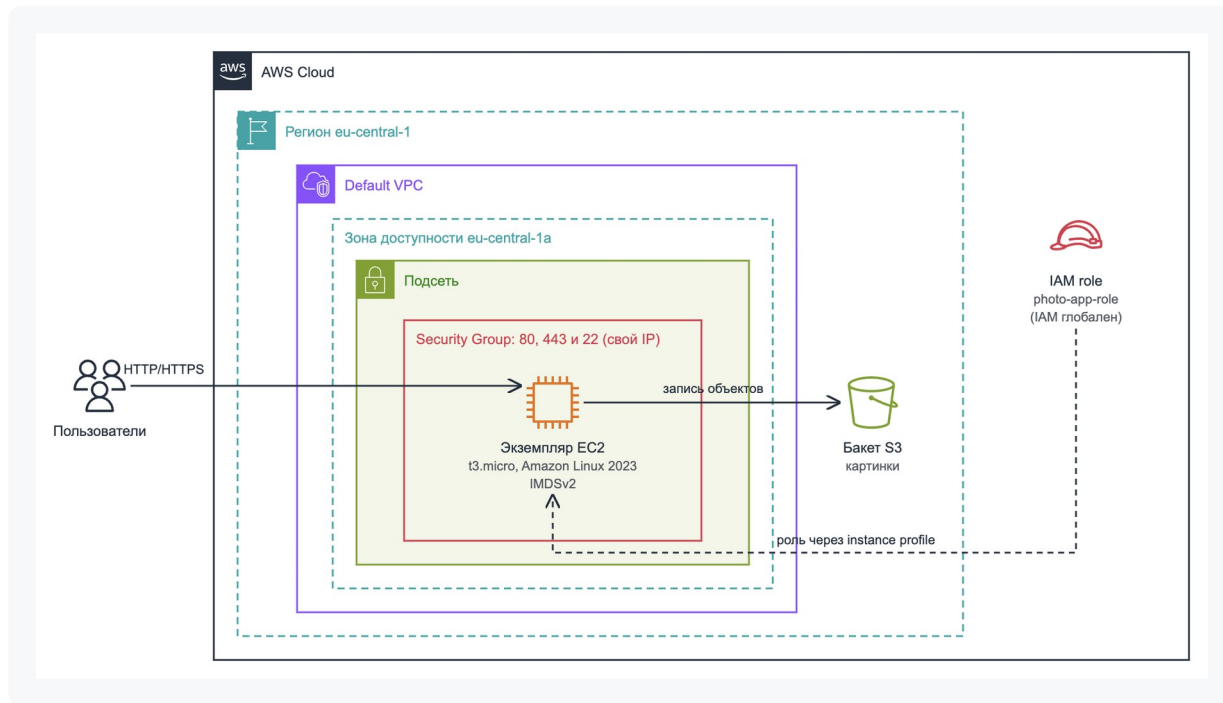


Рис. 4.8. Архитектура учебного сценария

Слабые места первой версии

- **Одна точка отказа.** Один экземпляр в одной зоне: упала зона, упало приложение.
- **Нет масштабирования.** Придут все одноклассники сразу, и `t3.micro` не справится.
- **Меняющийся адрес.** После остановки ссылку придётся рассылать заново.
- **Данные на сервере.** База на том же экземпляре разделит его судьбу.

Каждый пункт закроет следующий блок: своя сеть в нескольких зонах, балансировщик и масштабирование, управляемая база данных, мониторинг.

Резюме (1 из 2)

- Приложение собирается из **строительных блоков**. Начинаем с вычислений: остальные блоки подключаются к ним.
- **Amazon EC2** даёт виртуальные машины. Сервис региональный, каждый экземпляр зональный: **один экземпляр - одна точка отказа**.
- Экземпляр собирается из **AMI, типа, хранилища, сети, Security Group, пары ключей и user data**. Имя m7g.large кодирует семейство, поколение, опции и размер.
- **EBS** переживает остановку, **instance store** стирается.
- Security Group пропускает только **явно разрешённое** и запоминает соединения. SSH не открывают для всего интернета.

Резюме (2 из 2)

- За работающий экземпляр платят посекундно, за остановленный нет, но **EBS** и **Elastic IP** оплачиваются, пока существуют.
- Программам нужна **machine identity**: роль IAM через instance profile даёт **временные ключи** из IMDS.
- **IMDSv2** защищает ключи от кражи через SSRF, а **минимальные права** ограничивают ущерб, если кража произошла.
- Модель оплаты выбирают по предсказуемости нагрузки: **On-Demand** для неопределённости, **Savings Plans** для постоянной, **Spot** для прерываемой.

Библиография (1 из 2)

1. Amazon Web Services (2026). *What is Amazon EC2?* Amazon EC2 User Guide. Available at: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html>
2. Amazon Web Services (2008). *Amazon EC2 Exits Beta and Now Offers a Service Level Agreement*. What's New with AWS.
3. Amazon Web Services (2026). *Amazon EC2 instance type naming conventions*. Available at: <https://docs.aws.amazon.com/ec2/latest/instancetypes/instance-type-names.html>
4. Amazon Web Services (2026). *Key concepts for burstable performance instances*. Amazon EC2 User Guide.
5. Barr, J. (2023). *New - AWS Public IPv4 Address Charge + Public IP Insights*. AWS News Blog. Available at: <https://aws.amazon.com/blogs/aws/new-aws-public-ipv4-address-charge-public-ip-insights/>
6. Amazon Web Services (2026). *Connect to a Linux instance using EC2 Instance Connect*. Amazon EC2 User Guide.
7. Amazon Web Services (2026). *Amazon EC2 instance state changes*. Available at: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-lifecycle.html>
8. Amazon Web Services (2026). *Amazon EC2 Pricing*. Available at: <https://aws.amazon.com/ec2/pricing/>
9. Meli, M., McNiece, M.R. and Reaves, B. (2019). *How Bad Can It Get? Characterizing Secret Leakage in Public GitHub Repositories*. NDSS 2019.

Библиография (2 из 2)

10. Amazon Web Services (2026). *AWSCompromisedKeyQuarantineV3*. AWS Managed Policy Reference Guide.
11. Amazon Web Services (2026). *IAM roles for Amazon EC2*. Available at: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/iam-roles-for-amazon-ec2.html>
12. U.S. Department of Justice (2019). *Seattle Tech Worker Arrested for Data Theft Involving Large Financial Services Company*.
13. Novaes Neto, N. et al. (2020). *A Case Study of the Capital One Data Breach (Revised)*. MIT Sloan. Available at: <https://web.mit.edu/smadnick/www/wp/2020-16.pdf>
14. Office of the Comptroller of the Currency (2020). *OCC Assesses \$80 Million Civil Money Penalty Against Capital One*. Available at: <https://www.occ.gov/news-issuances/news-releases/2020/nr-occ-2020-101.html>
15. AWS Security Blog (2019). *Add defense in depth against open firewalls, reverse proxies, and SSRF vulnerabilities with enhancements to the EC2 Instance Metadata Service*.
16. Amazon Web Services (2026). *Spot Instance interruption notices*. Amazon EC2 User Guide.
17. Amazon Web Services (2026). *What are Savings Plans?* Savings Plans User Guide.
18. Amazon Web Services (2026). *Reserved Instances for Amazon EC2 overview*. Amazon EC2 User Guide.